MAD 1 WEEK 5

Week 5: Controllers - Business Logic

MVC Origins

Model-View-Controller (MVC) is a **design pattern** used for organizing code, primarily in GUI and web applications.

- First introduced in **Smalltalk-80** for GUI applications.
- Promotes **separation of concerns**, dividing an application into:
    1. **Model** (Data and Business Logic)
    2. **View** (User Interface)
    3. **Controller** (Handles User Input & Interaction)
- While originally intended for GUI applications, MVC concepts have been adapted to web applications.
- Web servers are **stateless**, meaning they do not maintain the state of the client between requests.

Controllers: Handling Actions

- Controllers handle **user interactions**, serving as the intermediary between the Model and View.
- They **receive user input**, process it, update the Model, and **send updates to the View**.
- Controllers help maintain a **clear separation** between UI and data, improving modularity and maintainability.

Actions in a Controller

1. **Receiving User Input** → The controller takes user input from the View.
2. **Updating the Model** → It processes the input and updates the Model accordingly.
3. **Updating the View** → The View is refreshed to reflect any changes in the Model.

Requests and Responses

Web applications work on a **Request-Response** cycle:

- **Client (Browser) sends a request** to the server.
- **Server processes the request** and sends a response.
- Types of Requests:
    - **GET** → Retrieve information (e.g., visit a webpage)
    - **POST** → Submit data (e.g., form submission)
    - **PUT** → Update existing data (e.g., edit a record)

- **DELETE** → Remove data (e.g., delete an entry)

API Request & Response

Request:
`GET https://www.example.com/api/posts?category=technology&limit=2`

Response:

```
[
  {
    "id": 123,
    "title": "Sample Post",
    "content": "This is the content of the post."
  },
  {
    "id": 124,
    "title": "Another Post",
    "content": "More content here."
  }
]
```

HTML webpage request & response

Request: GET https://www.example.com/blog
GET /index.html HTTP/1.1
Host: www.example.com

• In this GET request, the client is requesting an HTML webpage from the server using the GET method.
• The base URL is https://www.example.com , and the endpoint is /blog .
• The server will respond with the HTML content of the webpage.
• The default file name is index.html , so the server will look for a file named index.html in the /blog directory.

Response:
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 100

```
<html>
<head>
<title>My Website</title>
</head>
```

```
<body>
This is my website.
</body>
</html>
```

CRUD Operations

CRUD stands for **Create, Read, Update, Delete**—the fundamental operations for handling data.

1. **Create** → Add a new entry (e.g., new student, course, etc.).
2. **Read** → Retrieve and display data (e.g., student details, grade reports, etc.).
3. **Update** → Modify existing data (e.g., updating marks, address changes, etc.).
4. **Delete** → Remove existing data (e.g., remove graduated students, delete a course, etc.).

CRUD in Databases:

- Some databases are **Read-heavy** (e.g., analytics dashboards), while others are **Write-heavy** (e.g., transaction logs).
- CRUD provides a **structured approach** to API design and database interactions.

API

- API stands for Application Programming Interface.
- It is a set of definitions and protocols that allow two applications to communicate with each other.
- APIs are used to share data and functionality between applications.
- APIs provide a standardized interface, abstracting the underlying complexity of systems.
- APIs follow a request-response model, where one application sends requests to another to access data or perform actions.
- There are many different types of APIs, such as Web APIs, Library APIs, Database APIs and Hardware APIs.
- Web APIs include REST APIs, RPC APIs, GraphQL APIs, etc...

API for CRUD

- An API for CRUD is a set of endpoints that allow users to perform CRUD operations on a database.
- These endpoints are typically exposed via HTTP requests, and they use the HTTP methods GET , POST ,PUT , and DELETE to represent the different CRUD operations.

GET

• The GET is used to retrieve data from the database.

• Example:

◦ GET /students - This endpoint would return a list of all students in the database.

POST

• The POST is used to create new data in the database.

• Example:

◦ POST /students - This endpoint would create a new student in the database.

PUT

• The PUT is used to update existing data in the database.

• Example:

◦ PUT /students/1 - This endpoint would update the student with the ID of 1.

DELETE

• The DELETE is used to delete data from the database.

• Example:

◦ DELETE /students/1 - This endpoint would delete the student with the ID of 1.

Controllers & Business Logic

- **Controllers group related actions together**, such as:
    - Handling user authentication.
    - Sending emails or notifications.
    - Logging activities.
- Example: Laravel PHP framework uses Controllers to manage groups of related actions.

Routes and URL Mapping

- **Routes map URLs to actions in a Controller.**
- HTTP Verbs (GET, POST, PUT, DELETE) define what action a request should trigger.
- URLs convey context and determine which function to execute.

Example in Python using **Decorators**:

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper
# Applying decorator to a function
@my_decorator
def greet():
    print("Hello")

greet()
```

Out:
Before function call
Hello
After function call

Summary

- **Controllers** manage interactions between the Model and View.
- **Actions** represent user-triggered operations.
- **Requests and Responses** define how the web operates (stateless system).
- **CRUD operations** are essential for data management.
- **Routes and Decorators** help map URLs to specific actions in web frameworks.

SQLAlchemy(Screencast):

Basic flask:

```
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///students.db'
db = SQLAlchemy(app)
```

```python
@app.route('/')
def home():
    return render_template('home.html')

if __name__ == '__main__':
    app.run(debug=True)
```

Create model:

```python
class Student(db.Model):
 id = db.Column(db.Integer, primary_key=True)
 name = db.Column(db.String(50), nullable=False)
 grade = db.Column(db.Integer, nullable=False)
```

DONT FORGET:

```python
with app.app_context():
   db.create_all()
```

Query data:

In app.py

```python
@app.route('/students')
def students():
    students = Student.query.all()
    return render_template('students.html', students=students)
```

Populate the db:

```python
# Run in Python shell
from app import db, Student
student1 = Student(name="Alice", grade=90)
db.session.add(student1)
```

```
db.session.commit()
```

Jinja to display:

<!-- templates/students.html -->

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Students</title>
</head>
<body>
    <h1>Student List</h1>
    <ul>
        {% for student in students %}
            <li>{{ student.name }} - Grade: {{ student.grade }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

Filter:

```
top_students = Student.query.filter(Student.grade > 80).all()
```

Update in main.py(diff file):

```
from app import db,Student,app
with app.app_context():
    student1 = Student(name="hgvu", grade=100)
    db.session.add(student1)
    db.session.commit()
```

Deleting

```
@app.route('/delete/<int:id>',methods=['GET','POST'])
def delete(id):
   student=Student.query.get(id)
   db.session.delete(student)
   db.session.commit()
   return render_template('home.html')
```

Html:

```
<ul>
        {% for student in student %}
        <li>{{ student.id }}</li>
       <li>{{ student.name }}</li>
       <li>{{ student.grade }}</li>

    <a href="{{url_for('delete',id=student.id)}}">Delete</a>
       {% endfor %}

   </ul>
```

JINJA DOCS: https://jinja.palletsprojects.com/en/stable/
SQL DATABASE VIEWER APP: https://sqliteviewer.app/
SQLALCHEMY DOCS: https://docs.sqlalchemy.org/en/20/